Raspberry Pi Inference Mesh

Akash Premkumar
28 September 2026

Abstract. Language-model chat usually ends at a cloud completions API. This paper treats the place that call ends, a hosted model or a machine the caller can name, as part of the surface, and it describes one local answer. Pi 0.2 High puts a standard-library Python router in front of a three-node Raspberry Pi mesh and exposes `POST /v1/chat/completions` on port 18080, with a small chat page on the same port. We hold the request shape fixed, the fleet fixed (pi2 runs llama.cpp on port 8080; pi3 and pi4 run Ollama on port 11434; the default model tag is `qwen2.5:0.5b`), and the failure rule fixed: a request pinned to one peer does not fall through to another peer. What varies, and only when the mesh is allowed to choose, is which healthy peer answers. The router lives in `pi-pair/` at `https://github.com/akashnaren/raspberry-pi-fun`. This write-up contributes the system and the method for reading it. The fleet is being brought up for local models, so the paper does not report a benchmark grid.

1. Introduction

An agent that chats, or a person using a small chat page, sends one request and waits for text. In ordinary practice that request leaves the machine. It crosses a network to a completions API, a model the caller does not hold, and a bill the caller does not see until later. The program on the near side of the call can be careful about what it shows the model. It still cannot say which computer produced the tokens, or what happens when the path to that computer fails.

Cloud-only inference binds three ordinary properties of the loop.

1. **Placement.** The model is not on the hardware sitting next to the loop. A board that could run a small checkpoint stays idle, and every turn pays a round trip to someone else's machine.
2. **Naming.** The caller names a model id. It does not name a machine. When several machines could answer, the choice is hidden inside the provider.
3. **Failure.** If the path the caller had in mind is down, a hosted API may retry, reroute, or return a generic error. The caller cannot say "this board, and no other."

A local mesh puts those three properties back on the call. The request shape stays the one agents and chat clients already speak. The thing that changes is where the request is allowed to end.

Pi 0.2 High is that surface for one small fleet. Three Raspberry Pi boards, pi2, pi3, and pi4, sit in one custom 3D-printed vertical rack. Their Tailscale names are rpi-pi2, rpi-pi3, and rpi-pi4. A router written in the Python standard library exposes `POST /v1/chat/completions`, `GET /health`, and `GET /peers`, and serves a chat page from the same process. Behind the route, pi2 runs llama.cpp and pi3 and pi4 run Ollama. When a request omits a model, the tag is `qwen2.5:0.5b`.

The measurement this setup is for, and which this paper does not yet report, is a placement comparison. Hold the chat contract fixed. Hold the three-node fleet fixed. Hold the rule that a pin does not hop to a different board. A later run can then vary the checkpoint, the peer, and a cloud call, and read cost and reliability against a target that did not move. This note writes down that target. It is a system and a method. It is not a score table.

1.1 What is held fixed

- The client contract: chat completions, including a streaming mode that returns server-sent events.

- The peer roles: pi2 is llama.cpp on port 8080; pi3 and pi4 are Ollama on port 11434.
- The default model tag, `qwen2.5:0.5b`, when the request does not name one.
- The pin rule. A request that names a peer never selects a different peer.
- The implementation constraint. The router imports the standard library only.

1.2 What is allowed to vary

- Which healthy peer answers, when the caller lets the mesh choose. The router round-robins that list.
- The model string, when the caller sends one.
- Whether the response is one JSON object or a stream.
- Later, and not in this note: the checkpoint, a cloud condition, and a task set.

1.3 Contributions

1. A local chat surface with one completions route in front of a three-node Raspberry Pi mesh, so an agent or a browser can call the fleet the way it already calls a hosted chat API.
2. An explicit routing contract. The caller names a peer, or lets the mesh choose. Those two facts sit beside the body. If they arrive inside the JSON instead, they are removed before a worker sees the messages. A pinned peer that is down fails closed.
3. A bound on what the first write-up claims. The fleet is being brought up for local models. The checks that exist today exercise the router against stand-in workers. They are not a timing study of the boards.

Section 2 describes the fleet, the router, and the HTTP surface. Section 3 records the choices that keep the surface small and the pin honest. Section 4 shows the rack. Section 5 says what has been observed and what has not. Section 6 places the note among nearby work. Section 7 concludes.

2. System

The code is the `pi-pair/` tree. The name on the process and in this paper is Pi 0.2 High. One Python process serves the chat page and the proxy. The default bind is all interfaces on port 18080. Nothing in the router is installed from a package index.

2.1 Peers

A peer is a name, a host, a port, and a kind. The kind is Ollama or llama.cpp. The built-in map is the fleet the chat was written against.

Name	Kind	Port	Routes the router calls
pi2	llamacpp	8080	`GET /v1/models`, `POST /v1/chat/completions`
pi3	ollama	11434	`GET /api/tags`, `POST /api/chat`
pi4	ollama	11434	`GET /api/tags`, `POST /api/chat`

Hosts are not printed in this paper. The installer copies the example map onto a board only when that board does not already have a peer file. On Tailscale the boards are rpi-pi2, rpi-pi3, and rpi-pi4. Those names are how the fleet is addressed. They are not a claim about who else can open port 18080.

pi2 is also probed on port 11434 when port 8080 does not answer. That second try stays on the peer named pi2. It is not a hop to pi3 or pi4. Section 3.5 keeps the two cases apart, because they are easy to conflate.

2.2 The call

The process is a threaded HTTP server from the standard library. The chat page is served from the same process. A request that leaves the page directory, or that names a path with a slash, is not served. `GET /` and `GET /index.html` return the chat page with the default model tag substituted into the HTML. `GET /health` and `GET /peers` return the same document: whether the router itself is up, the model tag, how many peers probed successfully, the peer list, the inference-slot cap, and how long a health snapshot is kept. Router up and peer down are different facts in that document. Section 5 treats the split as an observation, not as a failed run.

`POST /v1/chat/completions` is the only body the router accepts, and it is the request the method holds fixed. The caller names a peer, or leaves the choice to the mesh. Those two facts sit beside the body. If they arrive inside the JSON instead, they are removed before a worker sees the messages. By default the mesh may choose. The model is whatever the request names, or else `qwen2.5:0.5b`. Temperature defaults to 0.7. The token cap defaults to 256.

Concurrent inference is limited to three slots. A request that cannot take a slot within 120 seconds is refused with HTTP 503. The slot is released when the handler returns, including when the worker failed.

A non-streaming success is a chat completion. It records which peer answered, how many milliseconds elapsed, the model string actually sent, and the worker kind. A streaming success is a sequence of events: a comment line naming the peer, a role chunk, content chunks, a final chunk carrying the same four facts, and the usual done marker. Worker failures become HTTP 502. Any other path is 404. The router is not an authentication boundary. Reaching port 18080 is enough to call it. The wire names for the routing facts and for those four recorded fields are written once, in the example at the end of this section.

2.3 Choosing a peer

The function that picks a peer is the rule the rest of the paper depends on.

If the mesh is off, or the caller has named a peer, the router looks up that name and stops. An unknown name is an error that includes the name. A known name whose probe failed is an error for that name, with the peer's note appended when it has one. A known name that is up, but cannot serve the requested model, is an error that says so. There is no second candidate. Mesh off does not mean "pick whoever is up." Asking the mesh to choose while the mesh is off is still a name lookup. No peer has that name, so the call errors.

If the mesh is on and the caller has not named a peer, the router keeps peers whose probe succeeded and whose advertised models can serve the request, then round-robins that list. An empty list is an error that no peer is up.

Usable is defined differently for the two kinds, and the difference matters for a later comparison. On an Ollama peer the requested tag is usable when the tag list is empty, when the tag is present, or when the name before the colon occurs inside a listed tag. An empty list is treated as usable because a probe can succeed without returning names. On a llama.cpp peer, any non-empty model list marks the peer usable, regardless of whether the requested tag appears. The string sent onward is the requested tag only when that tag is in the list and contains no colon. Otherwise the router sends the first id the server advertised. The default tag contains a colon, so a llama.cpp peer is normally called with its own first id, not with `qwen2.5:0.5b`. The completion records that substitution as the model string and the worker kind in the measurement contract of Section 2.2. A reader who keeps those two facts can see the substitution. A measurement that drops them cannot.

2.4 Health cache

Peer probes are cached for 2.5 seconds. The cache exists so a page polling `/health` does not stampede `/api/tags` or `/v1/models`. It is process-local. The auto path reads the snapshot, which may be the cached one. The pinned path probes that peer again when the request is handled, so a pin is not decided only by a snapshot taken for someone else's page load.

2.5 Bring-up

The boards are started by hand. The installer leaves an existing peer file in place, and the router listens on port 18080.

A bring-up probe then requests health on the local port, probes each peer at the path that matches its kind, and sends one non-streaming completion with the mesh allowed to choose. The probe is not a benchmark. A router with no model process behind it can still pass the first half. Health answers, and the peers are down. That state is expected while the fleet is coming up. It is not a model result, and this paper does not treat it as one.

Example

The sections above can be read without the wire names. This sketch is the one place they are written down. The elapsed time is a placeholder. It is not a measurement from the fleet. The first call names pi3 and turns the mesh off, so only that board may answer. If pi3 is down, the error names pi3. The second call lets the mesh choose. The completion still names the peer that ran, how long it took, the model string that was sent, and the worker kind. Host addresses live in `peers.json`. The board installer is `install.sh`.

```
POST /v1/chat/completions
X-Pi-Target: pi3
X-Pi-Mesh: off

POST /v1/chat/completions
X-Pi-Target: auto
X-Pi-Mesh: on

200
X-Pi-Peer: pi3
{"pi_peer":"pi3","pi_ms":"(elapsed)","pi_model":"qwen2.5:0.5b","pi_kind":"ollama"}
```

3. Design choices

3.1 Standard library only

The boards already have Python. A router that needs a package install is a second step, a second failure, and a second thing to keep aligned on three machines. HTTP, JSON, threads, and the URL library are enough to proxy chat. The tests stay inside that boundary. They are the standard library's `unittest`, with fake Ollama and llama.cpp handlers in the same process. A reader can run them without the rack and without a model file.

3.2 One client dialect, two worker dialects

The chat page and any agent that already speaks completions should not have to know which board runs which runtime. The router translates. An Ollama peer receives its own chat route, with messages, a stream flag, and options for temperature and the token cap. A llama.cpp peer receives the same completions route the client already speaks, with messages, a stream flag, temperature, and the token cap. Ollama's stream is

newline-delimited JSON. llama.cpp's stream is server-sent events. Both are turned into server-sent events for the client, in the chat-completions chunk shape, with the peer name attached. The client dialect stays one. The worker dialect stays whatever that board already runs. The paths themselves are in the peer table.

3.3 Routing sits beside the body

The caller names a peer, or lets the mesh choose, and says whether the mesh is allowed to choose at all. Putting those facts beside the body keeps them next to the request and out of the prompt. A caller that cannot set them beside the body may send the same two facts in the JSON. Those keys are removed before messages are forwarded, so a worker log of the body does not carry a field the worker does not understand. The response names the peer that ran. Routing is visible on the way in and on the way out. The example in Section 2 writes the names once.

3.4 A pin does not fall through

This is the choice the system is willing to be inconvenient about. If the caller names pi3 and pi3 is down, the error names pi3. The router does not try pi4, and it does not try a cloud model. Letting the mesh choose is the only path that may pick among peers. Treating a pin as a preference, with fallthrough as a kindness, would make a later measurement unreadable: a success labeled pi3 might have been pi4. The same reason a controlled comparison refuses to swap the action set under a condition applies here. The name on the request has to be the machine that ran it, or the caller has to leave the choice to the mesh and the response has to say which machine it was.

3.5 A second port is not a second peer

pi2 may answer llama.cpp on port 8080 and, at other times, an Ollama-shaped probe on port 11434. The health check tries the configured port and then the alternate ports recorded for the peer named pi2. The name in the result does not change. That retry is there because one board has been observed on more than one port. It is not permission to leave the board. Section 3.4 is about peers. This section is about ports on one peer.

3.6 A small cap, and a short cache

Three boards, and a page that polls health, are enough to crowd a small model or a tags endpoint. Three inference slots, a two-minute wait, then HTTP 503, is a blunt limit. A 2.5 second health cache is a blunt limit of the other kind. Neither is a scheduler, and neither is tuned against a measured knee in latency. Both exist so the first overload behavior is "busy" or "cached" rather than a stampede. Section 5 does not report how those knobs behave under real clients. They are part of the fixed system, not part of a result.

3.7 The page is not a second protocol

`GET /` is a small page for a person at the rack or on the tailnet. It calls the same completions route as an agent would. There is no separate chat dialect for the browser. If the page and an agent disagree, the disagreement is in the client, not in a second server.

4. Hardware

pi2, pi3, and pi4 share one custom 3D-printed vertical chassis. The boards are stacked so each bay still reaches the front connectors: Ethernet and USB, with a USB Wi-Fi adapter and an antenna where the radio is a dongle rather than a socket on the board. The three figures in this section are photographs of that chassis on a desk. They are the mechanical record for the note. They are not a wiring diagram, and they do

not replace the peer table in Section 2.1. Board names in the prose follow the peer map. The photographs are not labeled with those names, and they are not used to guess a different model number than pi2, pi3, and pi4.



Figure 1. The rack on a desk. Three Raspberry Pi boards in one 3D-printed vertical chassis. USB radios and antennas sit on the front of the stack.



Figure 2. A closer view of the same chassis. Ethernet, USB, and the antenna leads remain open on the bays.

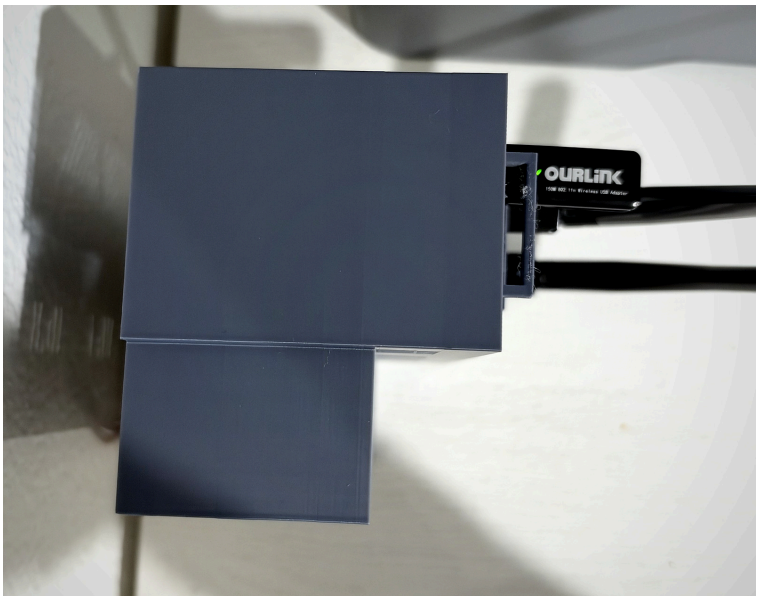


Figure 3. The top of the stack, photographed from above the desk. The upper board and its cooler are visible. The lower boards are the same chassis.

5. Status and early observations

This section is short because the fleet is being brought up for local models, and a short section is the honest one.

The router starts without a model process. `/health` can report the router up and a peer down in the same document. That split is the observation worth keeping. A green process and a down board are different facts, and the pin rule in Section 3.4 depends on not blurring them. The bring-up script prints both. While models are still being installed on the boards, "peers down" is a normal reading, not a failed experiment.

The unit tests stand up fake workers and check the contract: loading a peer list, the pin error when the named peer is down, assembly of a stream, and the shape of a completion. They do not boot a Raspberry Pi, they do not load `qwen2.5:0.5b`, and they do not time a token. A passing test supports Section 2. It does not support a sentence about how fast the rack talks.

The following are absent because they have not been measured on this fleet. Putting a number next to any of them would be an invention.

- Tokens per second, time to first token, or end-to-end latency, on any peer.
- A quality score, a task list, or a grader.
- A cloud model run as a paired condition against the same prompts.
- Behavior of the three-slot cap with real concurrent clients.
- Temperature, power, or throttling of the chassis under load.

One code fact should be recorded before those measurements start, because it will otherwise be misread as a result. The default tag is an Ollama-style name. On the llama.cpp peer the router usually substitutes the first id that peer advertises. A comparison of pi2 with pi3 under the default tag is not, by itself, a comparison of the same weights. The next measurement has to log the model string that was sent and the worker kind on every completion. Otherwise changing boards quietly changes models.

The method those runs should follow is the one this paper fixes, and it is the same shape as holding an interface constant while varying one factor. Same request shape. Recorded peer. Recorded model id. No silent substitution of a different board. A cloud call only when the protocol says the target is the cloud, not

as a hidden backup for a pin.

6. Related work

Local runtimes already exist, and this note uses two of them rather than replacing them. llama.cpp serves a model behind an OpenAI-compatible HTTP server. Ollama serves a model behind `/api/chat` and lists tags at `/api/tags`. Pi 0.2 High assigns llama.cpp to pi2 and Ollama to pi3 and pi4. It does not merge the two inside the worker. The single dialect is the caller's route.

That route is the chat completions shape already sent by agents, editor tools, and small chat pages. Keeping it means the mesh is a place a call can end, not a new client library. The hosted shape has no field for "which of my boards." The two facts in the example, which peer and whether the mesh may choose, are that field. They are local to this router. They are not proposed here as a standard.

`qwen2.5:0.5b` is a small Qwen2.5 tag, chosen as the default because a Pi-class board can hold a model of that size. This paper does not review the Qwen2.5 technical report, and it does not claim a quality number for the tag on these boards. The tag is fixed so that a request with no model field is the same request tomorrow.

Other on-device paths, including phone runtimes, vendor stacks, and a single board with a larger GPU, each answer "run a model here." The neighboring fact for this paper is narrower. A chat client already expects one HTTP completions endpoint. The work is to make that endpoint local, to name the board, and to fail closed when the named board is down. Systems that shard one large model across many machines are a different design. This fleet does not shard. Each peer that is up runs a whole small model, and the router picks a peer.

This note holds the request shape fixed and treats the place the model runs as the variable. What an agent is shown and where the model runs are easy to tangle. A structured view does not put tokens on a Pi, and a Pi mesh does not decide what an agent is allowed to click. They share the habit of writing down what is held still before claiming a comparison.

What this note does not claim is worth a paragraph of its own. That a small model can run on a small board is not news. That llama.cpp can speak chat completions is not news. That a pin ought to mean the named machine is a choice made in this router, not a result discovered about the industry. No citation is attached to that choice. The claim is the specific system: three named boards, two worker kinds, one completions route, a pin that does not fall through, and a written target for measurements that have not been taken yet.

7. Conclusion

Pi 0.2 High is a local chat surface for a three-node Raspberry Pi mesh. The caller sends a chat completion to port 18080. The router, standard-library Python in `pi-pair/`, proxies to llama.cpp on pi2 or to Ollama on pi3 and pi4, and defaults to `qwen2.5:0.5b` when the caller is silent about the model. The request says whether a named peer must answer. A named peer that is down stays down in the error. The call does not wander to another board, and it does not wander to a cloud model.

The rack in Section 4 is the hardware that map refers to. Sections 2 and 3 can be checked without it. The fleet is still being brought up, so this note stops before a table.